TraMask: Trajectory-Guided Unmasking of Behavioral Validation Gaps in Coding Agents

Research Proposal with a DeepSWE Replay Pilot

ANONYMOUS RESEARCHER

Coding agents validate their patches by generating and running tests. These tests often cover individual scenarios without checking whether the intended behavior remains observable under the context in which it matters. We study a precise form of this *combination gap*: *behavioral masking*. A test executes a target behavior, yet an unrelated context condition hides the effect of a fault in that behavior. As a result, correct and faulty implementations pass the same test.

We propose *TraMask*, a method that restores the missing observation with a small change to an existing test. It uses the coding agent's trajectory to identify the target test, recover task-relevant alternative implementations, and prioritize context changes. Paired executions then establish whether a candidate edit exposes a previously masked difference. Correctness is determined by an independent task oracle, not by assuming that the agent's final patch is correct. We give a concise problem definition, derive the algorithm from a running example, and specify replay-based pilot experiments on DeepSWE trajectories to test the assumptions before building the full system.

Additional Key Words and Phrases: coding agents, software testing, validation, behavioral masking, execution trajectories, test augmentation

1 Introduction

A coding agent resolves a software issue by inspecting a repository, changing code, running commands, and revising its patch in response to feedback. The agent often creates validation tests during this process. A passing test appears to support the patch, but this conclusion depends on the test's context and observation, not simply on reaching the relevant code. An empirical study of agent-generated tests reports that many agent-written checks emphasize observation rather than strong assertions [1].

The combination gap. Consider an issue that requires a message broker to stop forwarding messages after a hop limit. An agent tests ordinary forwarding with a configured downstream queue. It separately tests the hop limit with no downstream queue. Both tests pass. However, the hop-limit test also passes when the hop check is removed: without a destination, the message cannot be delivered regardless of the limit. The agent has tested two relevant features, but not the combination that makes the limit's effect visible.

This example exposes a concrete failure mechanism. A validation test can execute the intended feature while another context condition makes the observed result independent of that feature's correctness. We call this *behavioral masking*. Unlike a generic request for broader test coverage, the problem suggests a specific repair: find the context condition that suppresses the target effect and change it so that a relevant fault becomes observable.

Research question. *Can we automatically locate a masked target behavior in an agent-generated test and find a minimal, legal context change that exposes its effect?*

Coding-agent trajectories provide a practical way to address this question. They record the issue, code edits, test versions, execution commands, intermediate failures, and subsequent corrections. Such records can reveal both *what the agent tried to validate* and *which setup choices it considered*. Our hypothesis is that this task-specific history narrows the search for masking conditions, while program execution supplies objective feedback.

Existing mutation-based test amplification improves tests through program variants and test changes [2]. Infection and propagation analysis explains undetected mutants [3]. XAgent already

exploits trajectories to obtain relevant program context for additional validation tests [4]. TraMask focuses on a different, test-level operation: *find the specific context factor that hides a task-relevant behavioral difference, then repair that factor and exhibit the difference through paired execution.* The proposed pilot experiments will determine whether this operation occurs often enough and whether trajectories make it cheaper to automate.

2 Motivating Example

2.1 A forwarding-limit task

Suppose a message broker forwards a message only when (i) its dead-letter hop count does not exceed a limit H and (ii) a downstream queue is available. Write h for the current hop count and r for route availability. The delivery rule is

$$\text{Delivered}(h, r) = (h \le H) \wedge r. \tag{1}$$

The agent's task is to implement the hop-limit check. Equation (1) separates the *target condition*, $h \le H$, from the *context condition*, r. The example is deliberately small: the same logic appears in real workflows with routing, permissions, state initialization, and resource availability.

Listing 1. A complete, simplified message broker. The target behavior is the hop-limit check in `forward`.

```
H = 2

class Router:
    def __init__(self):
        self.queue = None

    def bind(self, queue):
        self.queue = queue

    def deliver(self, message):
        if self.queue is not None:
            self.queue.append(message)

def forward(message, hops, router):
    if hops > H:
        return
    router.deliver(message)
```

The correct implementation above returns early when $h > H$. A task-relevant faulty alternative, Q, omits the guard and always calls `router.deliver`. The final patch delivered by a coding agent is denoted P; in the general problem, P is *not* assumed correct. For the example alone, let P be the implementation in Listing 1 and Q the guard-omitting alternative.

2.2 Two passing tests, one missing combination

The agent checks ordinary forwarding with a bound queue at $h = H$, which passes. It also writes the following over-limit test:

Listing 2. The over-limit test never binds its queue, leaving the target effect unobservable.

```
def test_over_limit():
    queue = []
    router = Router()  # no downstream binding
    forward("blocked", H + 1, router)
    assert queue == []
```

Both checks pass on P *and* Q. The ordinary check does not exercise an exceeded limit. The over-limit test calls the target logic, but the missing queue binding prevents an attempted delivery from affecting the asserted queue. Table 1 makes the missing combination explicit.

Table 1. Results under the intended hop-limit behavior (P) and a guard-omitting alternative (Q). The last row supplies the missing combination.

Context	$h > H$	P delivers	Q delivers
Queue bound; ordinary hop count	No	Yes	Yes
Queue unbound; exceeded hop count	Yes	No	No
Queue bound; exceeded hop count	**Yes**	**No**	**Yes**

Key observation. The weak test reaches the hop-limit check but cannot reveal whether it works. Route unavailability suppresses the observable effect of removing that check. Binding the queue is sufficient to expose the fault; generating an entirely new test scenario is unnecessary.

2.3 A one-line unmasking edit

Keep the exceeded-hop input and the existing assertion. Add one setup operation:

Listing 3. A minimal context edit makes the original test sensitive to the missing guard.

```
def test_over_limit_unmasked():
    queue = []
    router = Router()
    router.bind(queue)  # unmask the target
    forward("blocked", H + 1, router)
    assert queue == []
```

Now P passes, while Q fails. A separate positive control with $h = H$ establishes that the configured route actually delivers messages. The unmasking edit and the positive control serve different purposes: the former exposes the target's effect; the latter validates the test setup.

This case is a controlled illustration rather than an observed result from the DeepSWE pilot. The pilot in Section 5 asks whether analogous masking conditions can be recovered automatically from real agent trajectories.

3 Problem Abstraction

3.1 Target behavior, context, and observation

An agent-generated test typically contains a setup, an operation, and an assertion. Let b be the behavior that the test intends to validate. Let P be the agent's delivered program, and let Q be a *task-relevant alternative* obtained from an earlier patch, a local reversion, or a targeted code change. The alternative is a probe, not a presumed incorrect implementation.

Let c denote the test's input and execution context, including fixtures and relevant system state. We distinguish two outcomes:

- $\mathsf{Eff}_b(V, c)$ is the effect of target behavior b when running version V in context c; for the broker, it records whether forwarding is attempted after the hop check.
- $\mathsf{Obs}_t(V, c)$ is the outcome visible at test t's observation point; for the broker, it is the content of the downstream queue.

Both quantities are measured at logical observation boundaries shared by the two versions. They do not require identical source-level control flow. A guard can be removed in one version while both executions are still compared at the ensuing forwarding decision.

Definition 1 (Behavioral masking). A test t has a behavioral masking witness for target b, versions (P, Q), and legal context c_0 when the target effects differ but the test-observable outcomes agree:

$$\mathsf{Eff}_b(P, c_0) \neq \mathsf{Eff}_b(Q, c_0) \quad \wedge \quad \mathsf{Obs}_t(P, c_0) = \mathsf{Obs}_t(Q, c_0). \tag{2}$$

The instrumented executions must establish that the target's effect is evaluated, rather than merely inferred from a passing assertion.

This definition excludes two neighboring cases. If neither execution reaches the target, the test *bypasses* it. If test-visible outcomes already differ but the assertions accept both, the test has an *oracle weakness*. Behavioral masking refers to the loss of a target effect *before* it reaches the relevant observation.

Definition 2 (Unmasking edit). A context edit δ transforms the setup of t from c_0 to $c_1 = c_0 \oplus \delta$. It is an *unmasking edit* if c_1 is legal for the issue, retains the same target behavior, and makes the two versions observably different:

$$\mathsf{Obs}_t(P, c_0 \oplus \delta) \neq \mathsf{Obs}_t(Q, c_0 \oplus \delta). \tag{3}$$

We seek a low-cost edit, measured initially by changed setup operations and, as a tie-breaker, by the number of paired executions. Minimality is judged among the explored legal edits; no global optimum is claimed.

In the broker, b is the hop-limit decision, c_0 has no route, and δ is `router.bind(queue)`. Crucially, (3) establishes a *behavioral contrast*, not its correctness direction. An independent task oracle determines whether P, Q, or both violate the requirement.

3.2 Research problem

Input: an issue I, an agent trajectory $\mathcal{H}$, the delivered patch P, and tests that the agent generated or executed. **Output:** a target–test pair (b, t), a task-relevant alternative Q, a legal small context edit δ, and reproducible paired executions establishing Equations (2) and (3). When a trustworthy task oracle is available, the output also includes a corrected assertion and a verdict about the delivered patch.

This formulation avoids a full decomposition of the issue into every possible requirement combination. It searches for a single behavior and a single context condition whose interaction explains an existing blind spot.

4 TraMask: Trajectory-Guided Behavioral Unmasking

TraMask follows a simple principle: *change the target, then change the context.* The first change asks whether an existing test reacts to a task-relevant variation of the target behavior. The second asks which small context edit exposes that variation. The trajectory identifies promising targets and contexts; paired execution decides which candidates work.

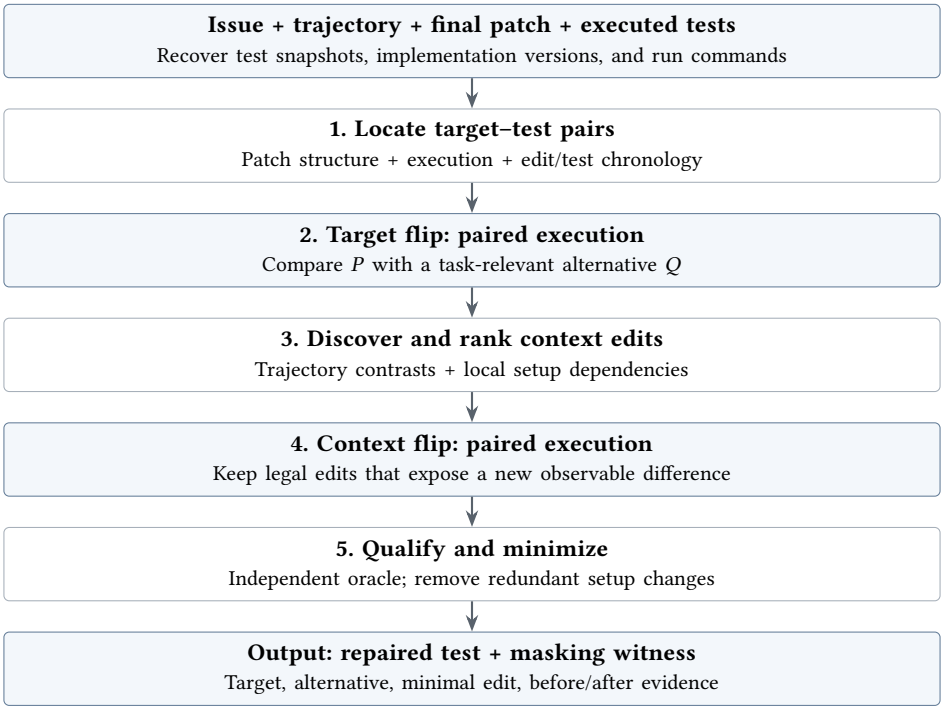


Fig. 1. TraMask's five-stage workflow. The central operation is a local context edit validated by two executions, rather than unconstrained generation of new tests.

4.1 Recover and localize the target

A trajectory contains more than the final patch. A test can first appear in a temporary script, be revised after an observed failure, and disappear from the delivered diff. TraMask reconstructs test snapshots and associates each with the code version and command that actually ran. It then proposes target–test pairs using (i) changed functions and their call sites, (ii) the path exercised by the test, and (iii) the chronological link between code edits, test edits, and validation commands. Agent explanations are useful search hints, not proof of correct requirements.

For the broker, editing the hop guard and then creating `test_over_limit` links that test to a concrete behavior. A test that merely mentions the broker but never executes forwarding is rejected at this step.

4.2 Flip the target and measure sensitivity

TraMask searches the trajectory for an earlier implementation that differs at the target. It prefers a small, locally comparable historical edit. If none exists, it reverses the relevant patch hunk or constructs a targeted mutation. The delivered patch is not the ground truth: both versions are treated symmetrically until an oracle is applied.

The current test runs on P and Q under the same restored context. Lightweight probes record the target effect and the public state consumed by the test. Equation (2) determines whether the pair supplies a masking witness. In the broker, the hop guard suppresses a forwarding attempt in P, but the absence of a bound queue makes both observed results empty.

4.3 Find a controllable masking condition

TraMask does not enumerate all possible environment states. It gathers a small set of edits from two sources. First, it mines *trajectory contrasts*: earlier test revisions, changed fixture calls, different object lifetimes, and setup choices associated with changed outcomes. Second, it inspects the local code between the target effect and the test's observation for guards, required resources, and state-dependent calls. Each candidate must be expressible as a change to the existing test setup.

For the broker, the target's forwarding attempt reaches `Router.deliver`, but the subsequent queue check discards the effect. The relevant setup operation is `bind`. The algorithm proposes binding the queue, not an unrestricted rewrite of the whole test. If no useful trajectory contrast exists, local program evidence supplies a fallback candidate.

4.4 Flip the context and qualify the test

Candidate edits are tried one at a time, ranked by direct trajectory evidence, closeness to the observation path, and edit size. For each edit, TraMask restores the same program versions, reruns the test, and checks Equation (3). It retains edits only when the target is still exercised and the context is valid. Successful edits are simplified by deleting unnecessary setup changes and rerunning the pair. Small two-edit combinations are a bounded fallback, not the default search space.

Finally, TraMask derives a verdict from a requirement, an independent verifier, or another trusted contract. It can report three outcomes: (i) *confirmed missed defect* if the delivered patch violates the contract; (ii) *validated stronger test* if the delivered patch passes and the alternative violates it; or (iii) *behavioral contrast only* when the correct outcome has not been established. The last outcome is evidence of missing sensitivity, not a confirmed software bug.

4.5 Algorithm

Listing 4 gives the complete loop. Both searches are restricted by the target and recorded execution; the final-patch verdict is determined separately.

Listing 4. TraMask: target flip, context flip, and independent qualification.

```
def tramask(issue, trajectory, final_patch):
    outputs = []
    for test, target in locate_pairs(trajectory,
                                     final_patch):
        for other in alternatives(target, trajectory,
                                  final_patch):
            before = twin_run(test, final_patch, other,
                              target)
            if not before.masked_effect:
                continue
            edits = context_candidates(test, target,
                                       trajectory, before)
            for edit in rank(edits):
                changed = edit_setup_only(test, edit)
                after = twin_run(changed, final_patch,
                                 other, target)
                if not (after.legal and
                        after.target_preserved and
                        after.obs_diff):
                    continue
                changed = minimize(changed, test,
                                   final_patch, other)
                verdict = independent_oracle(issue,
                                             changed, after)
                outputs.append((changed, edit,
                                before, after, verdict))
                break
    return outputs
```

5 Pilot Experiments: DeepSWE Trajectory Replay

The full algorithm should not be implemented before its two empirical assumptions are checked: (1) behavioral masking occurs in real coding-agent validation, and (2) trajectories carry useful clues for recovering a masking context. The pilot therefore emphasizes faithful replay and small controlled interventions rather than an end-to-end success rate.

5.1 Data and replay protocol

Starting corpus. The initial data source is a frozen collection labeled as *211 failed DeepSWE agent trials*. Each trial is expected to have an issue instruction, a final patch, and an agent trajectory containing tool actions and test attempts; the manifest audit will establish the actual count, unique tasks, models, and artifact completeness. Failed trials are a useful discovery pool but cannot estimate prevalence among all coding-agent runs. A small comparison set of successful DeepSWE trials will be added when available.

DeepSWE comprises 113 original repository-level tasks across five languages, with task-specific environments and behavior-oriented verifiers [5]. Its task repository provides the base commit, environment configuration, instruction, and separate verifier artifacts. DeepSWE v1.1 validates committed patches in a clean container [6]. These artifacts make controlled replay practical, while the agent's own validation commands and temporary tests remain separate from benchmark grading.

What replay means. Replay restores a recorded repository version and test snapshot, reruns a recorded command inside the pinned task environment, and compares its observable output with the archived tool output. It does *not* attempt to regenerate the model's hidden reasoning or reproduce its token-by-token decisions. For a test written at step k, replay must use the code state and test version at step k; a second run can apply that test to the delivered patch. This distinction prevents a historical PASS from being attributed to the wrong program version.

(1) **Index:** record trial ID, issue ID, base commit, runtime image, model, final-patch status, test-file snapshots, executed commands, and their outputs.
(2) **Restore:** reconstruct source edits and temporary scripts from the tool-action log; pin dependencies and use isolated, disposable environments.
(3) **Re-execute:** replay at least one agent-authored test at its original code state and then against the final patch; compare assertions, exit status, and relevant public state.
(4) **Qualify:** label missing files, nondeterministic behavior, environment drift, and replay mismatches. Such trials are not silently treated as negative examples.

The official task verifier is reserved for retrospective correctness checks and oracle qualification. Neither hidden tests nor reference solutions are given to the target locator or context generator. Verifier results provide evidence, not an infallible label; suspected verifier errors require independent adjudication.

5.2 Pilot 1: Are the alleged gaps genuine masking?

Question. How often does a generated test exercise a target behavior while hiding the observable effect of a task-relevant alternative?

Procedure. First audit a small number of readily replayable cases to stabilize the instrumentation, including a broker-like routing case and examples involving object identity or semantic conditions. Then sample 30 failed trials from the replayable pool without selecting for expected masking, plus 10 successful trials when available. Sample across distinct tasks where possible and report repeated trials from the same task as clustered observations. Two reviewers, blinded to the eventual automated repair, label each target–test pair as *masking*, *target bypass*, *weak oracle*, *valid discrimination*, or

insufficient evidence. For suspected masks, recover a local alternative and verify the two sides of Equation (2) by execution.

Observation sought. A collection of independently replayable cases in which a target-local effect differs while the original test's public observation is unchanged. Cases where the target never runs or an assertion is simply missing do not count as masking. Report the number of analyzable trials, target–test pairs, confirmed masks, and exclusions separately. This pilot will establish whether the proposed category is specific and empirically useful.

5.3 Pilot 2: Can trajectory history identify masking conditions?

Question. Do the agent's prior states and test revisions reveal controllable context factors that code-only analysis misses or ranks lower?

Procedure. For each confirmed mask, independently generate at most five candidate setup edits under three conditions: *test and patch only*, *test and patch plus local source context*, and *the same material plus chronological trajectory events*. Hold the language model, execution budget, and proposal count fixed. Measure whether the candidate set contains a manually validated masking condition and at what rank. Categorize the useful trajectory evidence: prior fixture revisions, failing/passing run contrasts, reused versus recreated objects, historical patch alternatives, or explicit test intent.

Observation sought. Lower rank or fewer paired runs to find a valid context edit when trajectory events are available. If trajectory access does not improve localization under equal budgets, use it only for target recovery; the method should not claim trajectory-guided masker discovery as its central contribution.

5.4 Pilot 3: Does a small context edit restore test sensitivity?

Question. Can a single setup change expose a meaningful behavioral difference while preserving test validity?

Procedure. For each confirmed mask, try the highest-ranked editable factor first. Run the delivered patch and alternative in matched clean environments. Require valid execution, continued target exercise, a new public observation difference, and a test setup that still represents a permitted use case. Remove redundant edits and rerun. Use an independent task contract or verifier to assign the correctness direction; if neither suffices, retain a contrast witness without claiming a defect.

Observation sought. The fraction resolved by one context edit, by two edits, and not resolved within the fixed budget; the distribution of edit sizes; and the number of distinct issues for which stronger validation or a previously missed defect is established. A mask that requires wholesale test regeneration does not support the proposed minimal-edit insight.

5.5 Pilot 4: Which information source is actually necessary?

Question. Is the useful signal produced by the two-flip procedure, the trajectory, or simply unconstrained test generation?

Procedure. On the same masked pairs, compare (a) direct language-model test augmentation, (b) target-specific mutation plus ordinary input/fixture variation, (c) the two-flip method without trajectory history, and (d) the full two-flip method with trajectory history. Keep the alternative programs, test budget, and runtime limits aligned wherever applicable. Record both successful and unsuccessful edits, not only the final winning attempt.

5.6 Decision criteria and pilot deliverables

The pilot is designed to make the research direction falsifiable. Three findings would justify building the full system: *(i)* reproducible masking instances occur in multiple independent DeepSWE tasks

Table 2. Pilot measurements and the evidence each one supplies.

Measure	Interpretation
Replay fidelity	Original command and output reproduced at the recorded code state
Confirmed masking pairs	Target-local difference and equal original observations both established
Masker rank / recall@k	Relevant setup factor found among a fixed number of candidates
Validated unmasking rate	Legal context edit causes a new observable difference with correct test semantics
Confirmed missed defects	Delivered patch fails an independently justified behavior check
Cost per valid witness	Paired executions, agent calls, and edited setup statements

rather than only a hand-picked example; *(ii)* a small, controllable setup change restores sensitivity in a substantial share of those cases; and *(iii)* trajectory history improves masker ranking or reduces paired executions at a matched budget. If masking is rare, the work should narrow its target class. If code-only context search performs equally well, the trajectory should become optional rather than a claimed source of novelty. If the distinctions are mostly oracle errors, the correct research problem is oracle improvement, not context unmasking.

The immediate deliverable is a compact *replay evidence set*: for every analyzed pair, the exact code and test versions, target anchor, alternative implementation, original paired observation, proposed context edit, new paired observation, and oracle verdict. This dataset will separate established observations from design assumptions and provide a rigorous basis for the subsequent full implementation.

References

[1] Zhi Chen, Zhensu Sun, Yuling Shi, Chao Peng, Xiaodong Gu, David Lo, and Lingxiao Jiang. 2026. Rethinking the Value of Agent-Generated Tests for LLM-Based Software Engineering Agents. *arXiv preprint* arXiv:2602.07900. https://arxiv.org/abs/2602.07900

[2] Benjamin Danglot, Oscar Luis Vera-Pérez, Benoit Baudry, and Martin Monperrus. 2019. Automatic Test Improvement with DSpot: A Study with Ten Mature Open-Source Projects. *Empirical Software Engineering* 24, 4 (2019), 2603–2635. https://doi.org/10.1007/s10664-019-09692-y

[3] Oscar Luis Vera-Pérez, Benjamin Danglot, Martin Monperrus, and Benoit Baudry. 2019. Suggestions on Test Suite Improvements with Automatic Infection and Propagation Analysis. *arXiv preprint* arXiv:1909.04770. https://arxiv.org/abs/1909.04770

[4] Hieu Huynh, Patanamon Thongtanunam, Michael Fu, Bach Le, and Kla Tantithamthavorn. 2026. XAgent: eXecution-guided Agentic AI for Effective Localization and Resolution of GitHub Issues. *arXiv preprint* arXiv:2609.09769. https://arxiv.org/abs/2609.09769

[5] Wenqi Huang, Charley Lee, Leonard Tng, and Serena Ge. 2026. DeepSWE: Measuring Frontier Coding Agents on Original, Long-Horizon Engineering Tasks. *arXiv preprint* arXiv:2607.07946. https://arxiv.org/abs/2607.07946

[6] Datacurve AI. 2026. DeepSWE Benchmark: Task Format, Environments, and Verifier Protocol. https://github.com/datacurve-ai/deep-swe